OP1 - Frameworks - B1 - Node.js

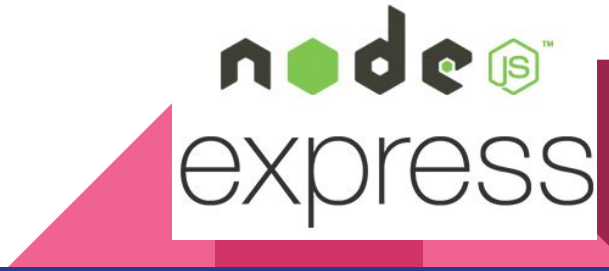
Sessió 04: Projecte Node.js amb Express

Express.js - El Framework Web per Node.js

🤔 Què és Express.js?

Express.js és un framework web minimalista i flexible per Node.js que proporciona un conjunt robust de característiques per desenvolupar aplicacions web i APIs.

Express és una capa **per sobre** del mòdul http que vam fer servir a la sessió 3. Per dins continua sent el mateix servidor, però ens estalvia feina.



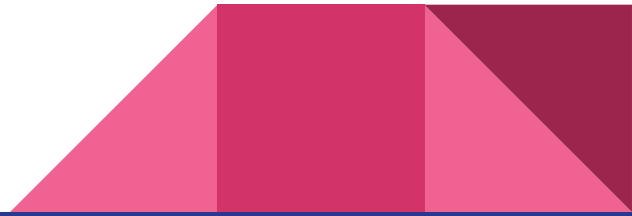
💡 Per què utilitzar Express?

Simplicitat

- Fàcil d'aprendre i utilitzar
- Codi més net i organitzat
- Menys línies de codi que amb Node.js pur

Característiques principals

- Routing avançat
- Middleware integrat
- Gestió de peticions i respostes més intuïtiva
- És el framework de Node més utilitzat



📊 Comparació amb Node.js pur

```
// Amb Node.js pur
const http = require('node:http');

const servidor = http.createServer((req, res) => {
    const url = new URL(req.url, `http://${req.headers.host}`);

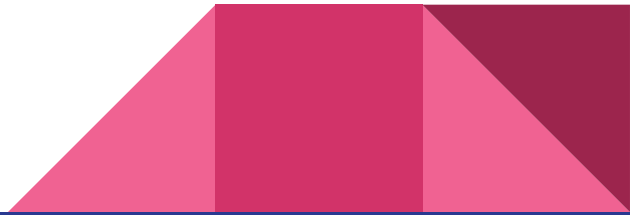
    if (url.pathname === '/api/usuaris' && req.method === 'GET') {
        res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ usuaris: ['Joan', 'Maria'] }));
    } else {
        res.writeHead(404, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ error: 'Ruta no trobada' }));
    }
});
```

Amb Express.js

```
// Amb Express.js
const express = require('express');
const app = express();

app.get('/api/usuaris', (req, res) => {
    res.json({ usuaris: ['Joan', 'Maria'] });
});

app.listen(3000);
```



canvis

Node pur	Express
`if (pathname === ... && method === ...)`	`app.get('/api/usuaris', ...)`
`writeHead` + `JSON.stringify` + `end`	`res.json(...)`
`new URL` per ignorar el `?`	Ho fa sol (`/api/usuaris?x=1` funciona)
`else` amb el 404 escrit a mà	Ho fa sol

🚀 Comencem amb Express, pas 1: Instal·lació

```
# Crear un nou projecte
mkdir projecte-express
cd projecte-express
npm init -y

# Instal·lar Express
npm install express
```

Què ha passat?

- 📄 `package.json` → ha aparegut:

```json
"dependencies": {
  "express": "^5.3.0"
}
```

- 📁 `node_modules/` → una carpeta nova amb Express i les seves dependències.
- 🔒 `package-lock.json` → guarda les versions exactes instal·lades.

⚠️ `node_modules` **no es puja mai a Git**. Crea un fitxer `.gitignore` amb:

```
node_modules/
```

Qui cloni el projecte només haurà de fer `npm install`.

Pas 2, primer servidor amb Express

Crea el fitxer `app.js`:

```js
// app.js
const express = require('express');
const app = express();
const PORT = 3000;

// Ruta principal
app.get('/', (req, res) => {
    res.send('Benvingut a Express! 🚀');
});

// Iniciar el servidor
app.listen(PORT, () => {
    console.log(`🌟 Servidor Express funcionant a http://localhost:${PORT}`);
});
```

Anem a pams (1/3), la preparació

```
const express = require('express');    // Importem Express
const app = express();                 // Creem l'aplicació
const PORT = 3000;                     // Port on escoltarà el servidor
```

- `require('express')` importa Express des de `node_modules`. No porta `node:` davant, perquè no és un mòdul intern de Node: l'hem instal·lat nosaltres.
- `express()` crea l'aplicació. A partir d'ara, tot es fa a través d'`app`.
- `PORT` és el port on escoltarà el servidor.

Anem a pams (2/3), la ruta

```
app.get('/', (req, res) => {
    res.send('Benvingut a Express! 🚀');
});
```

- `app.get()` defineix una ruta que respon a peticions **GET**.
- `'/'` és el camí de la ruta (aquí, la ruta principal).
- `(req, res) => {...}` és el **callback**: s'executa cada vegada que arriba una petició a aquesta ruta.
  - `req` és la petició (què ens demanen).
  - `res` és la resposta (què retornem).
- `res.send()` envia la resposta al client.

💡 `res.send()` detecta sol el tipus de contingut: amb un **text** envia `text/html`, i amb un **objecte** envia JSON. **Ja no cal el** `writeHead` **!**

Anem a pams (3/3), engegar el servidor

```
app.listen(PORT, () => {
    console.log(`🌟 Servidor Express funcionant a http://localhost:${PORT}`);
});
```

- `app.listen()` posa el servidor a escoltar al port indicat.
- El callback s'executa **una sola vegada**, quan el servidor ja està a punt.
- `console.log` ens confirma que tot funciona.

💡 És el mateix que el `servidor.listen()` de la sessió 3. Express no ens fa crear el servidor a mà amb `http.createServer()`: ho fa ell per dins.

Flux de l’aplicació

Quan executem node app.js (una sola vegada):

- Es carrega Express.
- Es crea l’aplicació.
- Es registren les rutes (encara no s’executen).
- S’inicia el servidor i es queda esperant.

Quan un client fa una petició (cada vegada):

- El servidor rep la petició.
- Express busca la ruta que coincideix (camí i mètode).
- S’executa el callback d’aquella ruta.
- S’envia la resposta al client.
- Si cap ruta coincideix, Express respon 404.

Provem el servidor

1. Configurem els scripts al `package.json`:

```json
"scripts": {
  "start": "node app.js",
  "dev": "node --watch app.js"
}
```

2. L'engeguem:

```
npm run dev
```

```
pilarmt@pc:~/Escriptori/lacetania 2026-2027/OP1/B2-RA2/express-01/projecte-express$ npm run dev

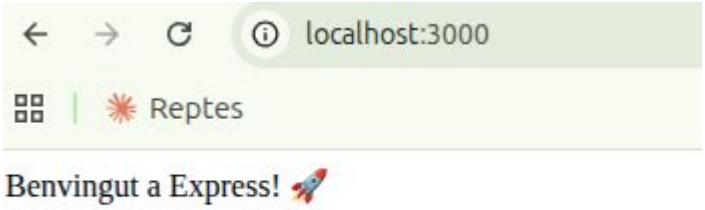
> projecte-express@1.0.0 dev
> node --watch app.js

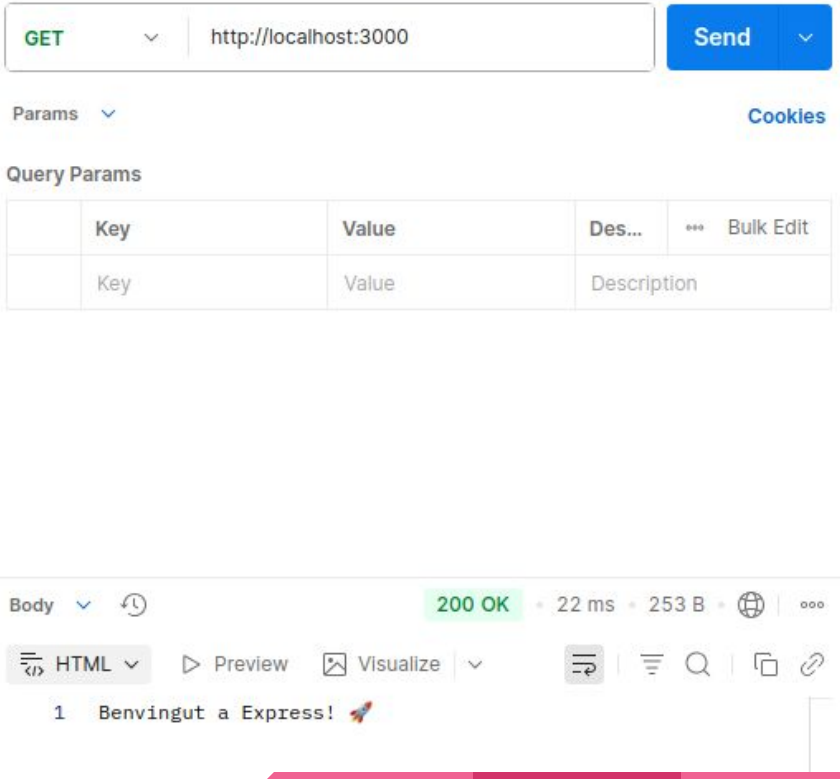
🌟 Servidor Express funcionant a http://localhost:3000
```

A Postman

🧪 **Ho comprovem al Postman i al navegador**

💡 Fixa't a Postman, a **Headers** de la resposta: `Content-Type: text/html; charset=utf-8`. **No l'hem escrit nosaltres**: l'ha posat `res.send()` automàticament.





Les nostres dades

De moment guardem les dades en un **array** dins del codi:

```
// Dades d'exemple

let productes = [

    { id: 1, nom: 'Portàtil', preu: 999 },

    { id: 2, nom: 'Telèfon', preu: 699 }

];
```

- Fem servir `let` (i no `const`) perquè més endavant hi afegirem productes.
- ⚠️ Les dades viuen a la **memòria**: si reiniciem el servidor, es perden els canvis. Més endavant farem servir una base de dades.

Ruta per obtenir tots els productes

🛒 **GET /api/productes: tots els productes**

```
app.get('/api/productes', (req, res) => {

    res.json(productes);

});
```

- `res.json()` converteix l'array a JSON i posa el `Content-Type: application/json` automàticament.
- A la sessió 3 això eren tres línies: `setHeader`, `JSON.stringify` i `res.end`.

Ruta amb paràmetre, un sol producte

🔍 GET /api/productes/:id: un sol producte

```js
app.get('/api/productes/:id', (req, res) => {
    const producte = productes.find(p => p.id === parseInt(req.params.id));

    if (!producte) {
        return res.status(404).json({ error: 'Producte no trobat' });
    }

    res.json(producte);
});
```

Línia a línia:

- `:id` és un **paràmetre de ruta**. Si demanem `/api/productes/5`, Express posa `"5"` a `req.params.id`.
- `parseInt(...)`: el paràmetre sempre arriba com a **text** (`"5"`), però els `id` de l'array són **números** (`5`). Amb `===`, `"5" === 5` és `false`.
- `.find()` busca el primer producte que compleix la condició. Si no en troba cap, retorna `undefined`.
- `if (!producte)`: si no l'ha trobat, responem **404**.
- `return`: atura la funció aquí. Sense el `return`, el codi continuaria i intentaria enviar una segona resposta (i Express donaria error).

💡 Recordeu la calculadora de la tasca 3? Allà vau haver de fer `split('/')` a mà. Amb Express, `/api/calculadora/:operacio/:num1/:num2` i ja ho teniu a `req.params`.

http://localhost:3000/productes

Save | Share

GET http://localhost:3000/productes Send

Params

Query Params

Key	Value	Des...	Bulk
Key	Value	Description	

Body 200 OK 11 ms 311 B

JSON Preview Visualize

```
[
    {
        "id": 1,
        "nom": "Portàtil",
        "preu": 999
    },
    {
        "id": 2,
        "nom": "Telèfon",
        "preu": 699
    }
]
```

http://localhost:3000/productes/1

Save | Share

GET http://localhost:3000/productes/1 Send

Params Cookies

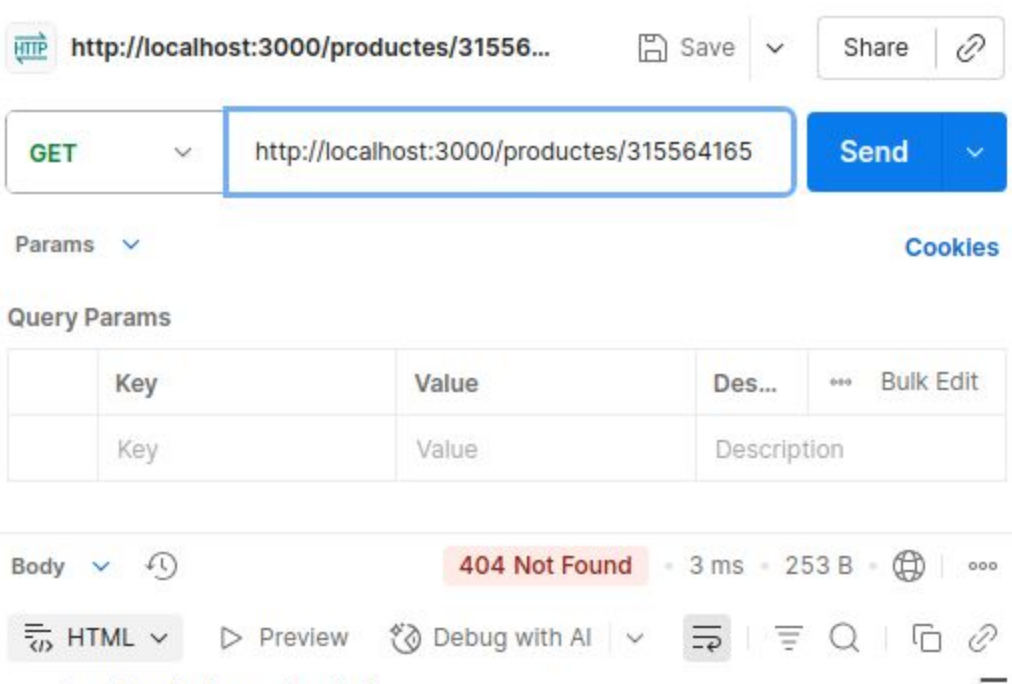
Query Params

Key	Value	Des...	Bulk Edit
Key	Value	Description	

Body 200 OK 4 ms 272 B

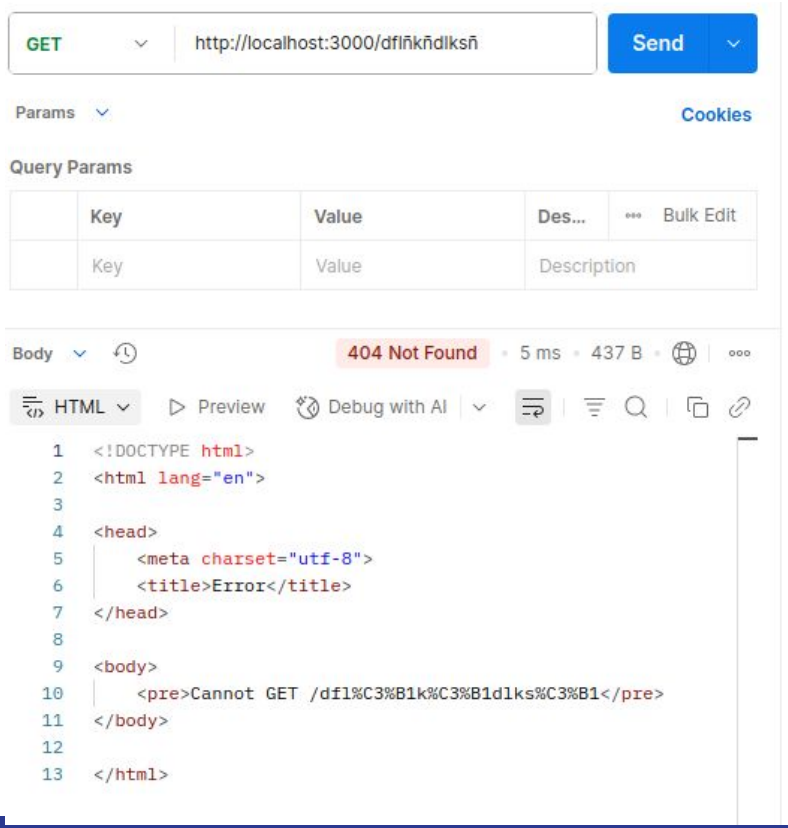
JSON Preview Visualize

```
{
    "id": 1,
    "nom": "Portàtil",
    "preu": 999
}
```


http://localhost:3000/productes/31556...
Save
Share
GET
http://localhost:3000/productes/315564165
Send
Params
Cookies
Query Params
Key
Value
Des...
Bulk Edit
Key
Value
Description
Body
404 Not Found
3 ms
253 B
HTML
Preview
Debug with AI
1 Producte no trobat

Què passa si demanem una ruta que no existeix?

🤔 Si som una **API JSON**, el client espera JSON, no HTML.

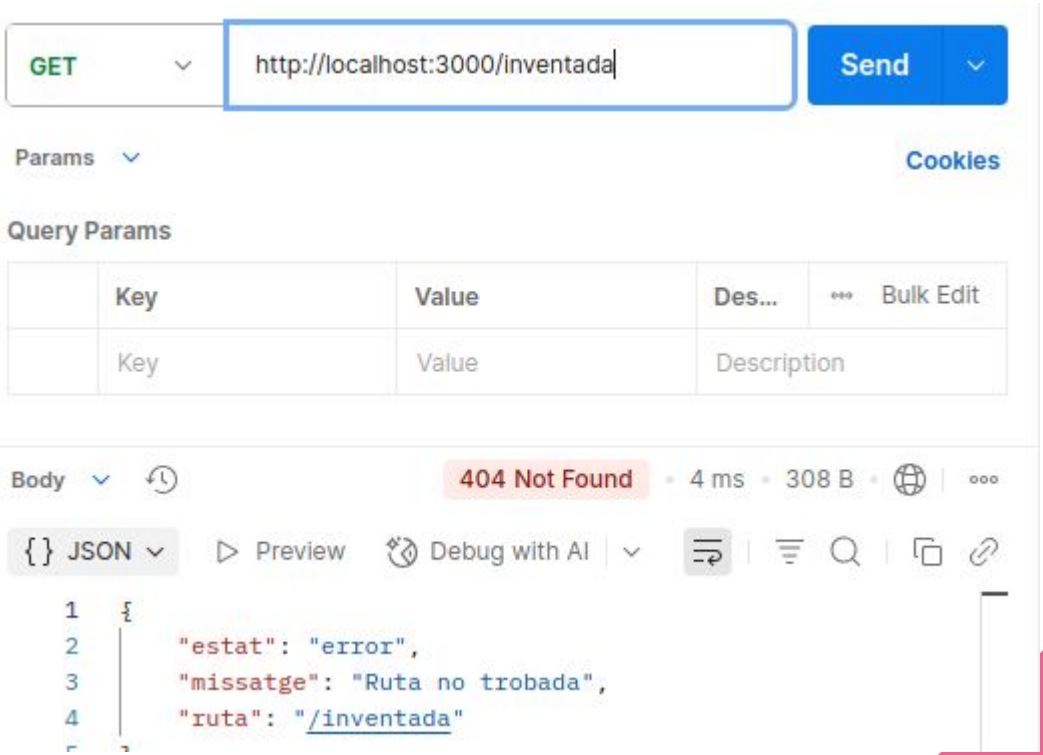


El nostre 404 en JSON

🚫 **Ruta 404: ha d'anar DESPRÉS de totes les rutes**

```
// Ruta 404 - Ha d'anar DESPRÉS de totes les altres rutes
app.use((req, res) => {
    res.status(404).json({
        estat: 'error',
        missatge: 'Ruta no trobada',
        ruta: req.path
    });
});
```

- `app.use()` sense camí s'executa per a **qualsevol** petició i **qualsevol** mètode.
- Per què **al final**? Express mira les rutes **en ordre, de dalt a baix**. Si aquest bloc anés el primer, respondria 404 a tot, fins i tot a les rutes que existeixen.
- `req.path` és el camí que ens han demanat, sense el `?`. És l'equivalent del `url.pathname` de la sessió 3.


GET
http://localhost:3000/inventada
Send
Params
Cookies
Query Params
Key
Value
Des...
Bulk Edit
Key
Value
Description
Body
404 Not Found
4 ms
308 B
JSON
Preview
Debug with AI
{
"estat": "error",
"missatge": "Ruta no trobada",
"ruta": "/inventada"

Middleware, express.json()

Per **rebre** dades amb POST, primer hem de preparar Express perquè sàpiga llegir JSON:

```
// Middleware per processar JSON (va ABANS de les rutes)
app.use(express.json());
```

Què és un middleware? Una funció que s'executa **entre** l'arribada de la petició i la nostra ruta:

```
Petició  →  express.json()  →  app.post(...)  →  Resposta
                  ↓
      llegeix el body, fa JSON.parse
      i el deixa a req.body
```

💡 `express.json()` fa **tota la feina dels chunks** de la sessió 3:

rebre els trossos, ajuntar-los, convertir-los a text i fer el `JSON.parse`.

POST /api/productes

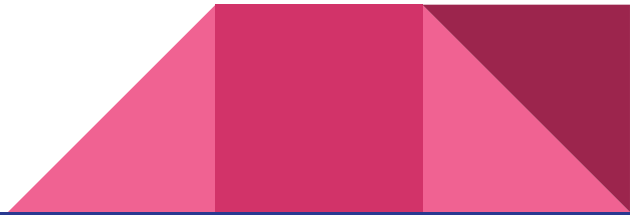
```
let seguentId = 3;   // El proper id que farem servir

app.post('/api/productes', (req, res) => {
    const { nom, preu } = req.body;

    // Validació: comprovem que ens han enviat les dades
    if (!nom || preu === undefined) {
        return res.status(400).json({
            estat: 'error',
            missatge: 'Cal indicar nom i preu'
        });
    }

    const nou = { id: seguentId++, nom, preu };
    productes.push(nou);

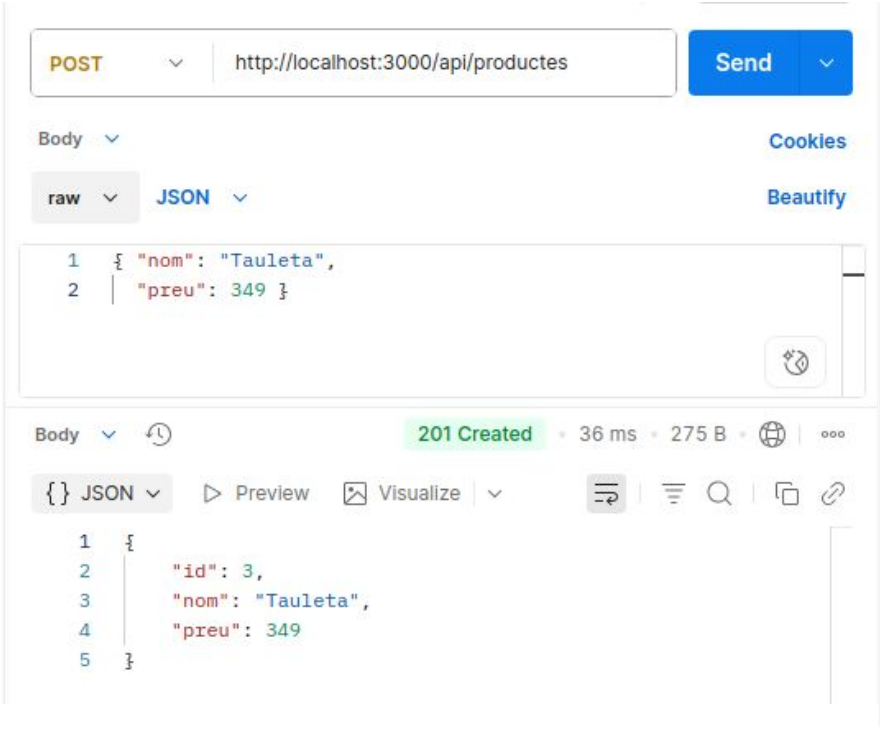
    res.status(201).json(nou);
});
```

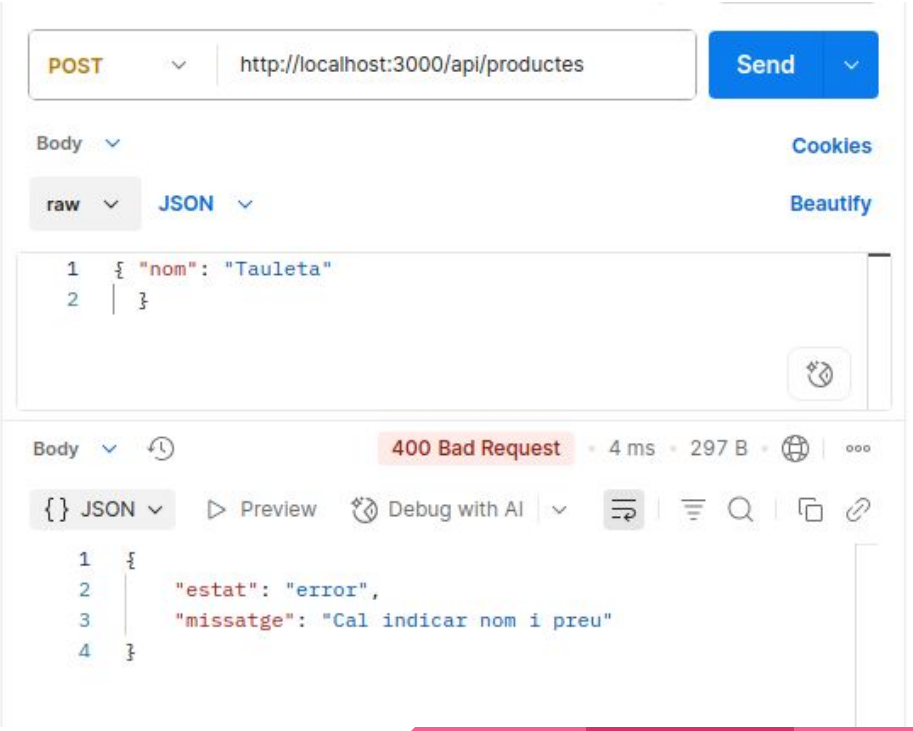


Línia a línia

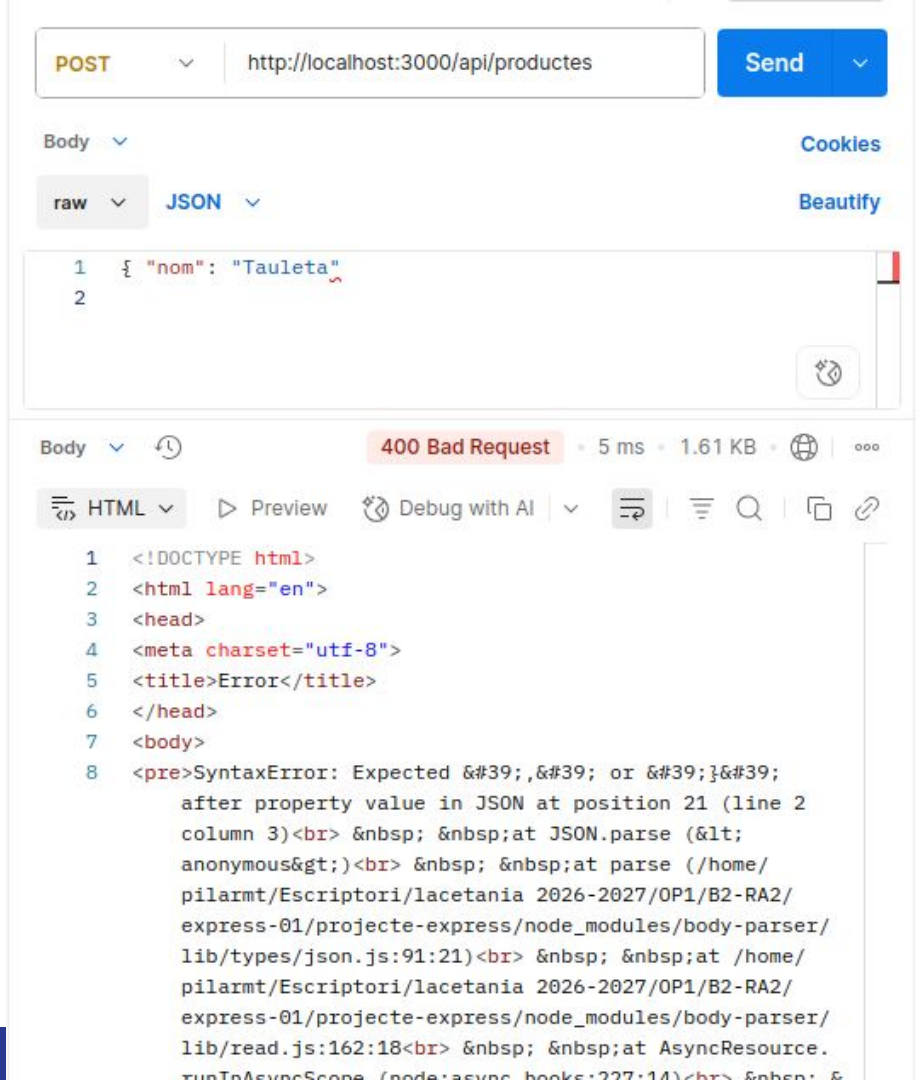
- `app.post()` és com `app.get()`, però per a peticions **POST**. Fixa't que el camí és **el mateix** que el del GET: el que canvia és el mètode.
- `req.body` conté les dades que ens envia el client, ja convertides en objecte gràcies a `express.json()`.
- `const { nom, preu } = req.body` és **desestructuració**: equival a `const nom = req.body.nom; const preu = req.body.preu;`.
- **Validació:** si falta alguna dada, responem **400**, igual que a la tasca de la sessió 3.
- `preu === undefined` en lloc de `!preu`: amb `!preu`, un preu de `0` es consideraria "buit".
- `seguentId++` fa servir el valor actual (3) i després el suma (4).
- `{ id, nom, preu }` és una abreviatura de `{ id: id, nom: nom, preu: preu }`.
- Responem **201 Created** i retornem el producte creat, amb el seu id nou.

Per què `seguentId` i no `productes.length + 1`? És el que fan molts tutorials, però falla quan s'esborra un producte. Si tens els ids 1, 2 i 3 i esborres el 2, la longitud és 2 i el nou producte rebria l'id 3, que **ja existeix**. Amb un comptador, això no passa mai.


POST
http://localhost:3000/api/productes
Send
Body
Cookies
raw
JSON
Beautify
1 { "nom": "Tauleta",
2 "preu": 349 }
Body
201 Created
36 ms
275 B
{} JSON
Preview
Visualize
1 {
2 "id": 3,
3 "nom": "Tauleta",
4 "preu": 349
5 }

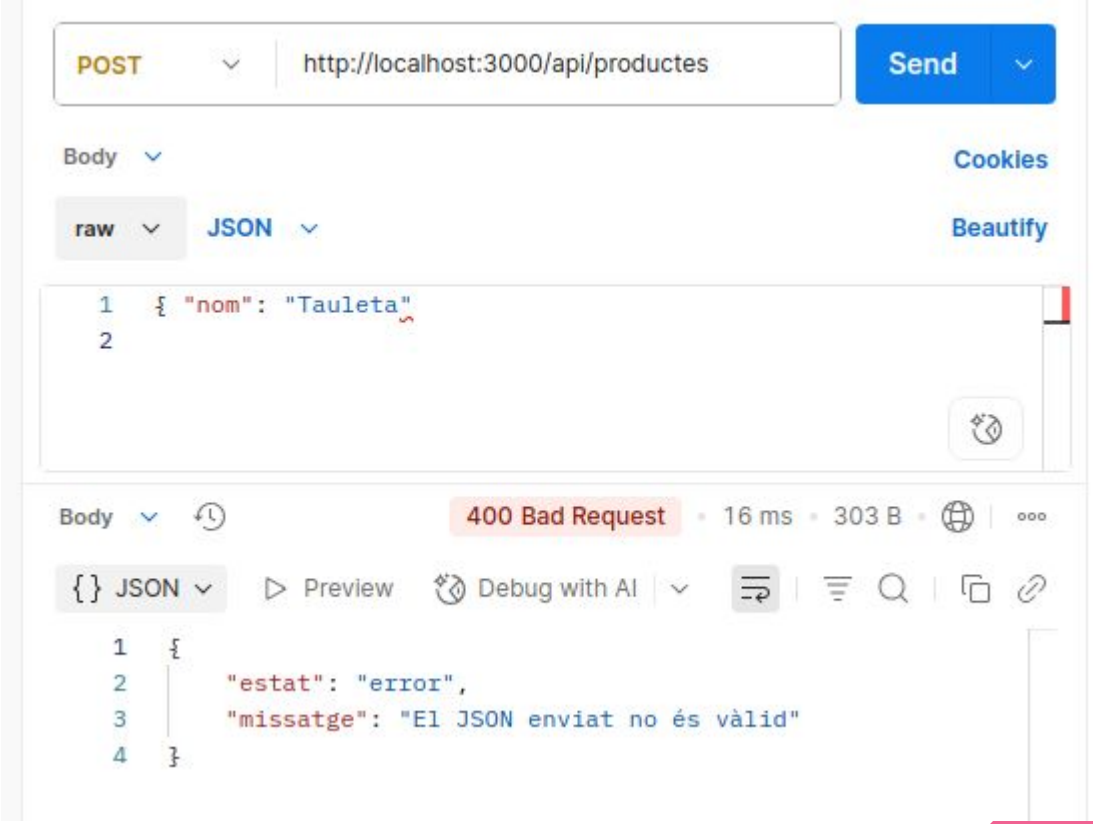

POST
http://localhost:3000/api/productes
Send
Body
Cookies
raw
JSON
Beautify
1 { "nom": "Tauleta"
2 }
Body
400 Bad Request
4 ms
297 B
{} JSON
Preview
Debug with AI
1 {
2 "estat": "error",
3 "missatge": "Cal indicar nom i preu"
4 }

JSON Mal format



Solució

```
61  // Gestió d'errors
62  // Gestor d'errors: va AL FINAL de tot, després del 404
63  app.use((err, req, res, next) => {
64      // JSON mal format (el detecta express.json())
65      if (err.type === 'entity.parse.failed') {
66          return res.status(400).json({
67              estat: 'error',
68              missatge: 'El JSON enviat no és vàlid'
69          });
70      }
71
72      // Qualsevol altre error inesperat
73      console.error(err);   // el detall només al terminal, per a nosaltres
74      res.status(500).json({
75          estat: 'error',
76          missatge: 'Error intern del servidor'
77      });
78  });
```


POST
http://localhost:3000/api/productes
Send
Body
Cookies
raw
JSON
Beautify
1 { "nom": "Tauleta"
2
Body
400 Bad Request
16 ms
303 B
{} JSON
Preview
Debug with AI
1 {
2 "estat": "error",
3 "missatge": "El JSON enviat no és vàlid"
4 }

🎯 Què hem après avui

Concepte	Exemple
Crear l'aplicació	`const app = express();`
Rutes per mètode	`app.get(...)`, `app.post(...)`
Paràmetres de ruta	`/api/productes/:id` → `req.params.id`
Dades del body	`app.use(express.json())` → `req.body`
Respondre JSON	`res.status(201).json(...)`
Ruta no trobada	`app.use((req, res) => ...)` al final
Gestor d'errors	`app.use((err, req, res, next) => ...)`

⚠️ **L'ordre importa:** middleware → rutes → 404 → errors → `listen`